

PORTA NUSA HOTEL

RFID Linen Visibility Platform

Solution FAQ

Benefit | Developer & Integration | Operation & Workflow
Partner Reference
28 June 2026

1. Solution Benefit FAQ

What problem does the solution solve?

Hotels can lose track of linen as it moves through storage, dispatch, return, and reconciliation processes. Manual counting is time-consuming, error-prone, and provides limited visibility until records are updated. Staff cannot easily identify which items were sent in a laundry batch or whether every item was returned.

This platform gives the hotel near-real-time visibility after each scan upload, a record of every laundry batch sent and returned, and an immediate view of items that are still outstanding at the vendor.

What is the benefit compared with manual counting?

Manual counting requires staff to physically handle and tally each item. With RFID, an operator waves a handheld reader over a pile of linen and captures every tag in seconds — without touching or unfolding each piece. The platform records the count automatically, time-stamps it, and links it to the operator's name and checkpoint.

Why use RFID instead of barcodes?

RFID tags do not need to be visible or individually aligned to be read. A single scan can capture multiple items in a bundle or laundry bag simultaneously. Barcodes require line-of-sight and must be read one at a time.

Can multiple linen tags be read at once?

Yes. The Chainway C5 E710 reads all UHF RFID tags within its antenna field simultaneously. During a single scan session, the device captures every unique EPC it detects, de-duplicates them, and reports the total unique count alongside raw read count. (CODE VERIFIED)

How does the platform help identify outstanding linen?

The Reconciliation view lists every active laundry batch that has outstanding items — meaning the count of items sent to laundry exceeds the count of items returned. Each batch row shows how many were sent, how many returned, and how many remain outstanding.

Does it support partial returns?

Yes. An operator can return a subset of items from a batch. The batch status shows **In Progress** while any items remain outstanding, and transitions automatically to **Completed** when all sent items have been returned. (CODE VERIFIED, PHYSICALLY VERIFIED)

Is there an audit trail?

Yes. Every scan session is recorded with the operator name, checkpoint, transaction type, timestamp, and the validation result for each EPC. The Transaction History page provides a full log of all sessions and their outcomes.

What is the purpose of Simulation Mode?

Simulation Mode allows the platform to be demonstrated without a physical RFID reader or real linen tags. A set of simulated linen items can be generated and used to walk through all three workflows. Simulation data is stored in an isolated database and has no effect on Hardware Mode data.

Can Simulation and Hardware data mix?

No. The two modes use completely separate SQLite databases (`simulation.db` and `hardware.db`). There is no shared state between them. A mode switch changes which database the platform reads and writes. (CODE VERIFIED)

Is the current solution already production deployed?

The solution is validated in a local demonstration environment. Production deployment is deferred and is not claimed as part of the current project scope. `DEPLOYMENT VERIFIED` is not claimed.

What business outcomes are demonstrated, and what is not yet claimed?

Demonstrated (PHYSICALLY VERIFIED):

- Dynamic laundry batch creation using any batch code

- Partial and full return workflows with correct status transitions
- `WRONG_BATCH` and `ALREADY_RETURNED` rejection enforcement
- `STOCK_COUNT` inventory recording
- Hardware and Simulation mode isolation
- Physical RFID scan with real tags on the Chainway C5 E710

Not yet claimed:

- Production deployment
- Formal ROI or time-savings calculations
- Integration with the hotel's existing PMS or ERP
- Guaranteed RFID read distance in meters

2. Developer & Integration FAQ

What is the high-level architecture?

```
RFID Tag
  → Chainway C5 E710 Android App (UHF RFID scan + HTTP POST)
    → POST /api/rfid/read-sessions (Next.js API route)
      → Backend Validation (business rules, EPC lookup, batch logic)
        → Hardware Database (SQLite via Prisma ORM)
          → Browser Dashboard (live data from the same database)
```

The Android app is the only RFID reader integration currently implemented. The backend handles all business logic; the Android app is a thin data-collection client. (CODE VERIFIED)

Does the Chainway C5 access the database directly?

No. The Android app communicates exclusively over HTTP. It posts a JSON payload to the web API, receives a validated JSON response, and has no direct database access.

What is the actual API endpoint used by the Android app?

```
POST {serverUrl}/api/rfid/read-sessions
```

(CODE VERIFIED — `MainActivity.kt` line 369 and line 590)

What HTTP method is used?

```
POST
```

What request headers are required?

Header	Value
<code>Content-Type</code>	<code>application/json</code>
<code>X-RFID-API-Key</code>	API key configured on the server
<code>X-Demo-Mode</code>	<code>HARDWARE</code> (hardcoded by the Android app)

(CODE VERIFIED — `MainActivity.kt` lines 377–379, 599–601)

How is authentication handled?

The Android app sends the API key in the `X-RFID-API-Key` header. The server compares the value against the `RFID_API_KEY` environment variable. Requests with a missing or incorrect key receive HTTP 401. There is no session token or OAuth flow. (CODE VERIFIED — `rfid-api.ts` lines 35–48)

What does the actual request payload look like?

```
{
  "clientId": "C5-HANDHELD-a1b2c3d4",
  "readerId": "C5-DEM0-01",
  "readerType": "HANDHELD",
  "operationMode": "MANUAL",
  "dataSource": "LIVE_DEVICE",
  "checkpoint": "LINEN_STORAGE",
  "transactionType": "SEND_TO_LAUNDRY",
  "laundryBatchCode": "LB-2026-001",
  "operatorName": "Demo Operator",
  "startedAt": "2026-06-28T09:00:00+08:00",
  "completedAt": "2026-06-28T09:02:30+08:00",
  "tags": [
    {
      "epc": "E28011700000020F6A2D9E4B",
      "rssi": -52,
      "readCount": 14,
      "firstSeenAt": "2026-06-28T09:00:05+08:00",
      "lastSeenAt": "2026-06-28T09:02:28+08:00"
    }
  ]
}
```

(CODE VERIFIED — `rfid-api.ts` lines 11–31, `MainActivity.kt` lines 607–643)

Field notes:

- `clientId` — UUID-based string generated by the app per session; used for idempotency
- `laundryBatchCode` — required for `SEND_TO_LAUNDRY` and `RETURN_FROM_LAUNDRY`; omitted for `STOCK_COUNT`
- `rssi` — the strongest RSSI value observed across all reads for that EPC during the session
- `readCount` — total times that EPC was read during the session; the backend expands this internally for deduplication accounting

What is the timestamp format?

ISO 8601 with timezone offset. Example: `2026-06-28T09:00:00+08:00`. The Android app uses `SimpleDateFormat("yyyy-MM-dd'T'HH:mm:ssXXX", Locale.US)`. (CODE VERIFIED — `MainActivity.kt` line 105)

What transaction types are supported?

Value	Description
<code>STOCK_COUNT</code>	Inventory count; no batch code required
<code>SEND_TO_LAUNDRY</code>	Dispatching linen to laundry; batch code required
<code>RETURN_FROM_LAUNDRY</code>	Receiving linen back from laundry; batch code required

(`ASSET_AUDIT` is recognized by the schema but returns a validation error — not implemented.) (CODE VERIFIED — `rfid-api.ts` lines 9, 143–148)

What are the batch-code rules?

Any non-empty string is accepted as a batch code. The system uses find-or-create logic: submitting a batch code that does not yet exist creates a new batch. Submitting an existing code adds to that batch. There is no enforced format; the hotel can use any naming convention. (CODE VERIFIED)

How does EPC handling work?

EPCs are normalized to uppercase and trimmed on receipt. Each unique EPC in the session is processed once. The `readCount` field in the tag object records how many times the reader physically detected that tag during the session.

How does deduplication work?

The Android app deduplicates tags within a single scan session before upload: each EPC appears only once in the `tags` array, with `readCount` reflecting total detections. At the server, each EPC is processed once per session. Submitting the same `clientId` again triggers an idempotent replay response rather than creating a duplicate session. (CODE VERIFIED — `MainActivity.kt` lines 507–531, `rfid-api.ts` line 73)

What is the retry and idempotency behavior?

If an upload fails (network error or HTTP error), the Android app transitions to ERROR state and shows a **RETRY UPLOAD** button. Pressing it re-submits the same session payload with the same `clientId`. The server detects the duplicate `clientId` and returns the original result without creating a new session or transaction. The response includes `"idempotentReplay": true`. (CODE VERIFIED)

What are the validation result codes?

Status	Meaning
ACCEPTED	EPC is registered and passed all business rules
UNKNOWN_EPC	EPC is not in the linen master
WRONG_BATCH	EPC exists but does not belong to the specified laundry batch
ALREADY_RETURNED	EPC has already been returned for this batch

(CODE VERIFIED — `rfid-api.ts` lines 174–179)

What does an actual response look like?

```
{
  "success": true,
  "idempotentReplay": false,
  "session": {
    "id": 42,
    "clientSessionId": "C5-HANDHELD-a1b2c3d4",
    "readerId": "C5-DEMO-01",
    "readerType": "HANDHELD",
    "operationMode": "MANUAL",
    "transactionType": "SEND_TO_LAUNDRY"
  },
  "summary": {
    "rawReadCount": 28,
    "uniqueEpcCount": 2,
    "registeredCount": 2,
    "unknownCount": 0,
    "acceptedCount": 2,
    "rejectedCount": 0,
    "duplicateCount": 0
  },
  "items": [
    {
      "epc": "E28011700000020F6A2D9E4B",
      "linenCode": "TWL-001",
      "linenType": "Towel",
      "status": "ACCEPTED",
      "reason": null,
      "readCount": 14
    }
  ],
  "transaction": {
    "id": 18,
    "transactionCode": "TXN-HW-0018",
    "transactionType": "SEND_TO_LAUNDRY"
  }
}
```

(CODE VERIFIED — `rfid-api.ts` lines 70–105)

Which component is the source of truth?

The web API and its database. The Android app submits raw scan data; all validation, business rule enforcement, and persistence happen on the server.

Is WebSocket used?

No. The Android app communicates via a single HTTP POST per session. The browser dashboard polls the server for updates. There is no WebSocket connection in the current integration. (CODE VERIFIED)

Could another RFID reader be integrated?

Architecturally yes, provided it can submit a JSON payload matching the schema above to the same endpoint with the required headers. The current physical integration has only been verified with the Chainway C5 E710. (NOT YET VERIFIED for other readers)

What has actually been physically verified?

The end-to-end hardware workflow — physical RFID trigger on the Chainway C5, EPC capture, HTTP upload, backend validation, and browser dashboard reflection — has been **PHYSICALLY VERIFIED** with real linen tags on the Chainway C5 E710, including `STOCK_COUNT`, `SEND_TO_LAUNDRY`, `RETURN_FROM_LAUNDRY`, partial returns, and rejection scenarios.

Production security recommendations

The following are architectural recommendations, not current claims:

- Replace the static `X-RFID-API-Key` with a short-lived token or mutual TLS for production
- Serve the web API over HTTPS only
- Restrict API access to known device IP ranges
- Rotate the API key on a scheduled basis
- Add rate limiting on the `/api/rfid/read-sessions` endpoint

None of these hardening steps have been implemented or penetration-tested in the current demo environment.

3. Operation & Workflow FAQ

How is a new physical EPC registered?

In Hardware Mode, open the **Hardware Linen Master** page in the browser. Any EPC that the Chainway has submitted and that is not yet in the master appears in an **Unknown EPCs** panel (polled approximately every 2.5 seconds). The operator enters the linen code and type for each unknown EPC and saves. From that point on, the EPC will receive `ACCEPTED` results in future scans. (`CODE VERIFIED`)

How does STOCK_COUNT work?

1. Open the Android app, select **STOCK_COUNT** from the workflow dropdown.
2. Enter the operator name and checkpoint. No batch code is needed.
3. Press **START** or press the physical trigger button. The reader captures all UHF tags in range.
4. Press **STOP** to end the scan. Review the tag list.
5. Press **CONFIRM UPLOAD** to submit. The server records the inventory count and returns a result for each EPC.

STOCK_COUNT does not create a laundry batch and does not trigger any inventory movement logic.

How does SEND_TO_LAUNDRY work?

1. Select **SEND_TO_LAUNDRY** and enter a batch code (e.g., `LB-2026-001`).
2. Scan the linen being sent to the laundry vendor.
3. Confirm and upload. The server creates the batch if it does not already exist, records each accepted item as sent, and marks the batch as **In Progress**.

How does RETURN_FROM_LAUNDRY work?

1. Select **RETURN_FROM_LAUNDRY** and enter the exact batch code used during SEND_TO_LAUNDRY.
2. Scan the returned linen.
3. Confirm and upload. Each item is validated against the batch. Items that were sent and not yet returned receive `ACCEPTED`. When all sent items have been returned, the batch transitions to **Completed** and disappears from the Reconciliation view.

What does partial return mean?

The hotel can return a subset of items from a batch. If 20 were sent and 12 are returned, the batch shows 8 outstanding and remains **In Progress**. The remaining 8 can be returned in a later session using the same batch code.

How does Reconciliation identify outstanding items?

The Reconciliation page shows every batch in the current mode where `accepted sent count > valid returned count`. The outstanding figure is calculated per EPC to handle repeated sends and returns correctly. When the outstanding count reaches zero, the batch is marked Completed and removed from Reconciliation automatically.

When does a batch become Completed?

A batch becomes **Completed** when all accepted sent items have valid matching returns and the outstanding count reaches zero. This is computed from transaction records rather than stored as a fixed status value.

What should the operator do for each error condition?

Condition	What it means	Recommended action
UNKNOWN_EPC	Tag scanned but not in the linen master	Register the EPC in the Hardware Linen Master, then re-scan
WRONG_BATCH	Tag exists but was sent under a different batch code	Verify the batch code entered on the device matches the batch the item belongs to
ALREADY_RETURNED	Tag has already been recorded as returned for this batch	No action needed; the item was already processed
Upload failure (network error)	No connection to the server	Press RETRY UPLOAD — the same session will be re-submitted idempotently
Reader initialization failure	RFID hardware did not initialize	Restart the app; check that the Chainway hardware is functioning
Duplicate tag reads	Same EPC read many times	Normal behavior; the app de-duplicates automatically
Incorrect batch code	Wrong code entered before upload	Clear the session, enter the correct batch code, and re-scan

How do power profiles work?

The Android app provides three read-range profiles:

Profile	Configured power level	Intended use
Near	5	Short-range precise scan (single item)
Medium	18	Standard operation (default)
Far	30	Wide-area scan (large pile or room sweep)

The profile is saved in device settings and applied before every scan. If the device fails to apply the selected power level, the scan is blocked and the status shows **FAILED TO SET POWER**. (CODE VERIFIED)

Why is exact read distance in meters not guaranteed?

RFID read range depends on tag orientation, tag placement on the fabric, nearby metal or liquid interference, and the specific antenna of the device. The power level controls transmit power, not a precise distance. The Near / Medium / Far labels describe relative range, not calibrated measurements.

What happens if network connectivity is interrupted during upload?

The Android app enters ERROR state and displays **RETRY UPLOAD**. The scan data is held in memory. Pressing Retry re-submits the identical session payload. The server uses the `clientSessionId` for idempotency: if the session was already recorded, the server returns the original result without duplication. If the app is closed before upload, the session data is lost and the operator must re-scan.

What does the browser RFID Scan page show?

In **Hardware Mode**, the RFID Scan page shows the most recent scan session submitted from the Chainway — the session ID, operator, transaction type, and per-tag results. It is a read-only view; the browser does not initiate or control hardware scans.

In **Simulation Mode**, the RFID Scan page is also read-only. It displays the last simulated session. Simulation data actions (generate, clear, reset) are performed from the Simulation Dashboard, not from this page.

What is the recommended event-demo sequence?

1. **Confirm the server is running** and accessible on the local network.
2. **Switch to Hardware Mode** in the browser.

3. On the Chainway C5, open the app and verify RFID Init shows **SUCCESS**.
 4. **Run a STOCK_COUNT** scan to show baseline inventory.
 5. **Run SEND_TO_LAUNDRY** with a batch code (e.g., `LB-EVENT-001`). Show the Laundry Batches page and Reconciliation view updating.
 6. **Run RETURN_FROM_LAUNDRY** with a partial subset to demonstrate **In Progress** state.
 7. **Run RETURN_FROM_LAUNDRY** for remaining items to demonstrate the batch reaching **Completed** and disappearing from Reconciliation.
 8. If an unknown tag is scanned, demonstrate EPC registration in Hardware Linen Master.
 9. **Switch to Simulation Mode** to show how the system behaves without physical hardware.
-

4. Known Boundaries

Confirmed

Capabilities implemented and verified in the current project:

- UHF RFID scanning with the Chainway C5 E710 handheld reader
- Three workflows: STOCK_COUNT, SEND_TO_LAUNDRY, RETURN_FROM_LAUNDRY
- Dynamic laundry batch creation using any batch code
- Partial and full return reconciliation
- Per-EPC validation with ACCEPTED / UNKNOWN_EPC / WRONG_BATCH / ALREADY_RETURNED status
- Idempotent session upload with retry support
- Dual isolated databases (simulation.db / hardware.db)
- Browser dashboard showing near-real-time inventory, batches, reconciliation, and transaction history after scan uploads
- In-app operator guides and demo operator checklist
- Read range / power profile control (Near / Medium / Far)
- Hardware database CLI maintenance scripts (init, backup, reset, verify)
- Web-first physical EPC registration with unknown-EPC live polling

Architecturally Possible but Not Yet Verified

- Integration with a different UHF RFID reader that can submit the same validated API payload
- Cloud or on-premises deployment is architecturally possible, but no production deployment model has been verified for the current project
- Multiple Android devices may be supportable through the same API architecture, but concurrent multi-device behavior has not been tested
- External system integration (PMS, ERP) via the existing REST endpoint

Not in Current Scope

The following are not part of the current demonstration and should not be presented as available:

- Production deployment or uptime guarantees
- Enterprise role-based access control (RBAC) or multi-user authentication
- Penetration testing or formal security assessment
- Formal SLA or support agreements
- Laundry-vendor portal or vendor-facing interface
- Advanced analytics or reporting beyond the current dashboard views
- Guaranteed RFID read distance in meters
- Automatic ROI or cost-savings calculation
- Partner-ready presentation deck (planned, not yet created)